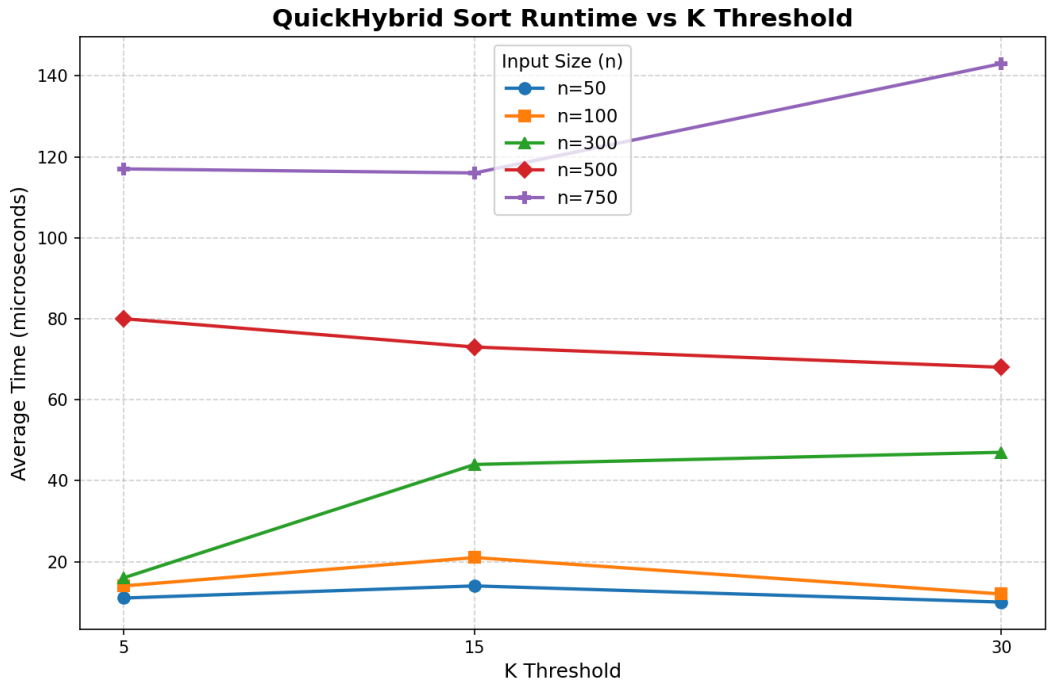
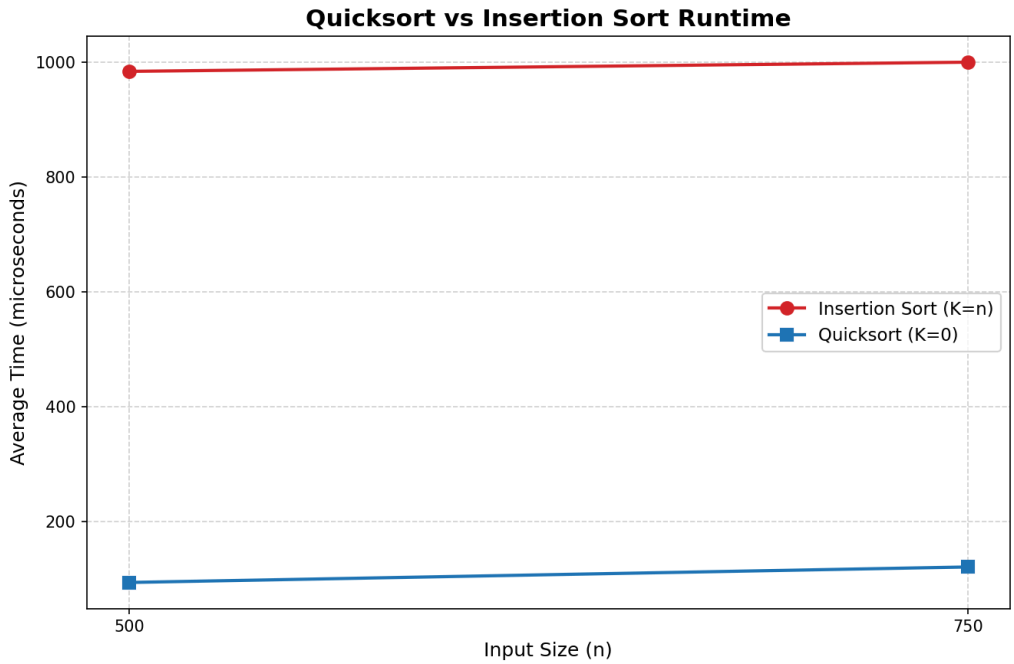
Algorithms Part 1

Graph for Task 2:



Graph for Task 3:

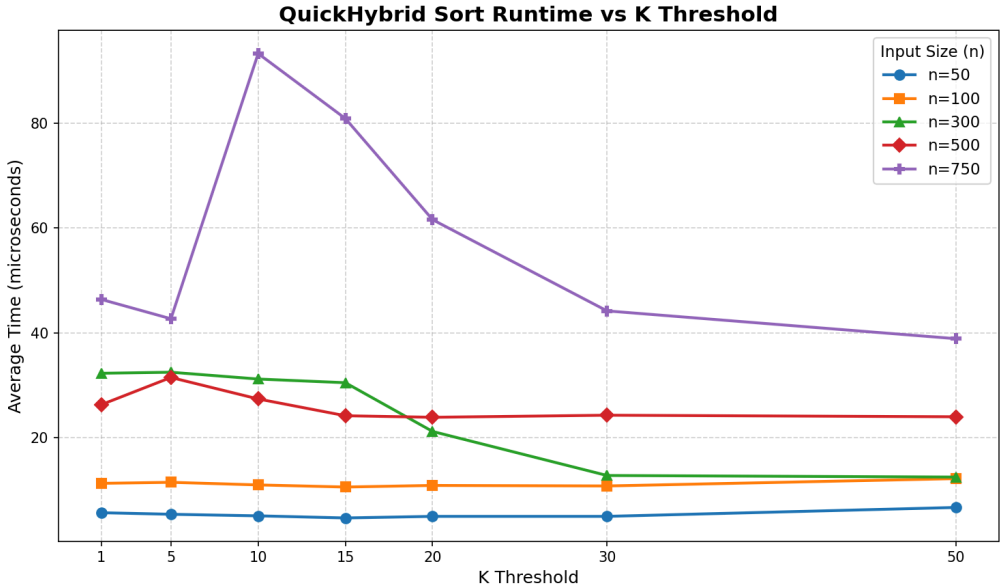


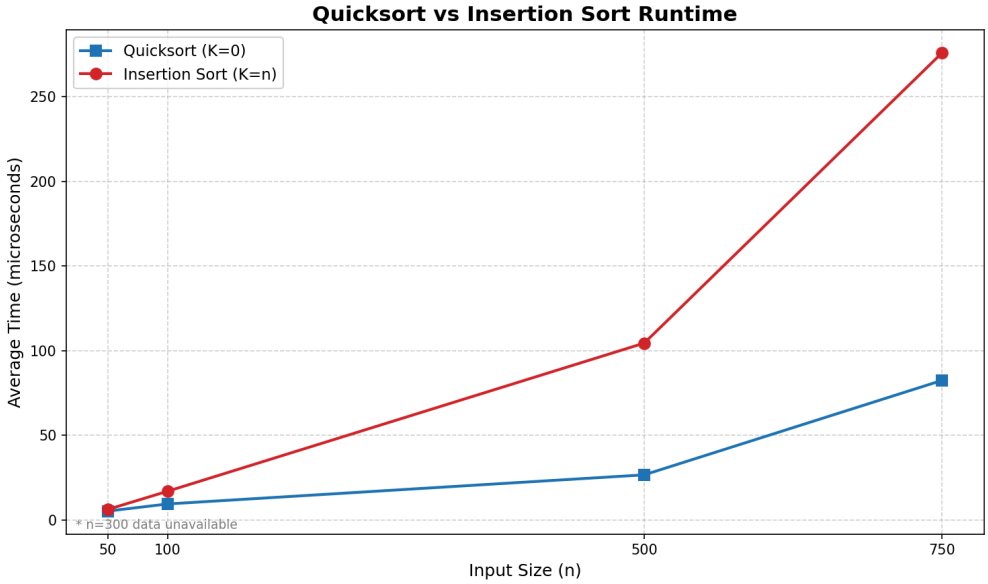
Deliverable 1.3: Optimal K-Value Observations:

- n = 50
  - When n = 50, the shortest time is produced by the QuickHybridSort, being 0.0000047s, whereas the QuickSort and InsertionSort produce a slower time with the optimal K value being K=15.
- n = 100
  - In this observation, the QuickHybridSort produced a slower time while QuickSort produced the fastest with K=15 still being the most optimal value. This suggests that the K-value should stay low for smaller arrays.
- n = 300
  - The most optimal K-value for n = 300 was K = 50, where QuickHybridSort was also the most optimal sorting method. Furthermore, the QuickSort was faster than InsertionSort because of the time complexity, whereas the InsertionSort is n^2 and QuickSort is nlogn which produces very different results when the number of n values are increased.
- n = 500
  - K = 20 was the most optimal for this set. As can be seen, the optimal k value is lower than the previous set despite the n being higher. Though this doesn't enforce a true pattern in our system, the differences in K values for n = 300 is within a millionth of a second, so it can be assumed the most optimal k lies within the 20-50 range for larger arrays.
- N = 750
  - The optimal K-value for this array was K = 50 where the QuickHybridSort produced a result more than two times faster than that of the pure QuickSort. In conclusion, when an array is larger, a larger K-value produces more accurate results.

Algorithms Part 1

Graphs for Task 4:





Observations:

For the hybrid sorting algorithm with varying k values, the graphs show how the hybrid sorting algorithm is only very effective with larger n values, as the arrays with smaller n values perform almost identically. For the larger n values, the runtime steadily increases as k increases, almost in a O(nlogn) complexity.

For the hybrid sort algorithm with the pre sorted inputs, at low n values, the k value does not make a very big impact on the runtime, but as you go up in n, the runtime decreases as n increases. For the pre sorted input in the quicksort and insertion sort algorithm, the runtime is shown to increase almost exponentially as n increases. The graph also shows how insertion sort is worse for larger input sizes and how quicksort is a lot better for the larger subarrays.